Разработка на LangGraph

Persistence & Async Scaling

Преподаватель

Андрей Носов

Опыт работы в NLP и ML 20 лет:

CTO / Ведущий ИИ Архитектор
@ Raft

Руководитель курса “AI-архитектор”
@ ОТУС

Образование:

2016 MSc Linguistics & AI
@KULeuven (Бельгия)

2020 PhD Communication Sciences
@UTA (Финляндия)

@AI_ARCHNADZOR

Зачем нужен Persistence?

ТРАДИЦИОННЫЙ LLM

Stateless

Обычный вызов API (OpenAI, Anthropic) не помнит предыдущие сообщения. Каждый запрос — как «чистый лист».

AI АГЕНТ

Stateful

Агент хранит историю сообщений, переменные и шаг выполнения. Это позволяет возобновлять работу в любой момент.

Ключевые сценарии 2026

1. Human-in-the-Loop (HITL)

Агент останавливается и ждет аппрува от человека (например, юриста). Ожидание может длиться часы или дни. Без персистенса процесс бы просто "умер" по таймауту.

2. Fault Tolerance (Отказоустойчивость)

Если сервер перезагрузился (deployment, сбой), агент не начинает заново, а загружает последнее состояние из БД и продолжает с прерванного шага.

3. Time Travel (Отладка)

Возможность "вернуться в прошлое": откатить состояние агента на 3 шага назад, изменить промпт или данные, и запустить ветку заново для исправления ошибок.

© 2026 GenSpark AI Slides

Механизм Checkpointing

Блок 1: Концепция

Snapshot после каждого шага

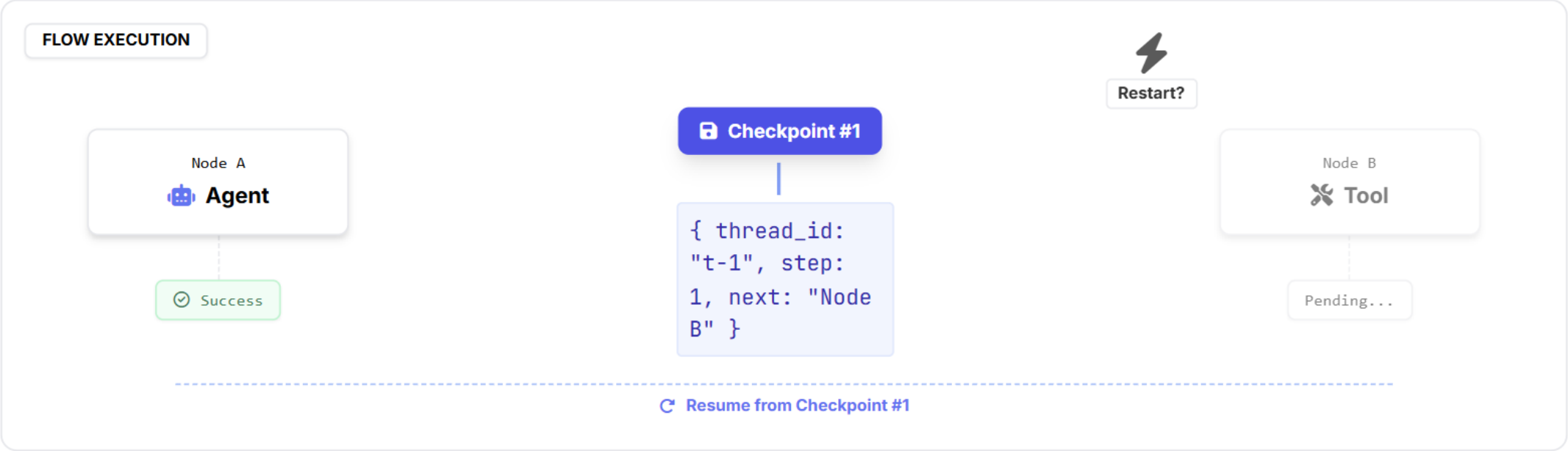
LangGraph автоматически сохраняет "снэпшот" состояния графа в базу данных после успешного выполнения каждого узла (Node execution). Это позволяет системе быть устойчивой к сбоям.

ЧТО СОХРАНЯЕТСЯ В CHECKPOINT?

State Values
Messages, Variables, Artifacts

Configuration
thread_id, checkpoint_id

Control Flow
Next Node (куда идти дальше)



Выбор бэкенда хранения (Storage)

DEV / TEST

MemorySaver

Хранит состояние в оперативной памяти (RAM) процесса. Данные теряются при перезагрузке.

USE CASES

- Юнит-тесты графов
- Одноразовые скрипты
- Демонстрации без БД

MVP / LOCAL

SqliteSaver

Легковесная файловая база данных. Обеспечивает персистенс без сложной инфраструктуры.

USE CASES

- Локальная разработка
- MVP прототипы
- Embedded-приложения

PRODUCTION STANDARD

AsyncPostgresSaver

Асинхронный драйвер для PostgreSQL. Использует пул соединений (psycopg) и JSONB для гибкого хранения состояния.

USE CASES

- Highload системы
- Enterprise-решения
- Долгосрочное хранение

© 2026 GenSpark AI Slides

Настройка Checkpointer

Блок 1: Концепция

server.py — Python 3.12

```python
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

# Инициализация пула (обычно при старте FastAPI / Lifespan)
async with AsyncPostgresSaver.from_conn_string("postgresql://user:pass@localhost:5432/pg_db") as checkpointer:

    # Компиляция графа С памятью (внедряем зависимость)
    app = builder.compile(checkpointer=checkpointer)

    # Запуск с уникальным thread_id (ключ сессии)
     config = {"configurable": {"thread_id": "ticket-12345"}}
     await app.ainvoke(inputs, config=config)
```

Connection Pooling
AsyncPostgresSaver использует пул соединений (через psycopg 3) для эффективной работы под нагрузкой.

Compile Injection
Передача checkpointer при компиляции превращает граф из Stateless в Stateful автоматически.

Thread Isolation
thread_id изолирует память разных пользователей или процессов (тикетов) друг от друга.

© 2026 GenSpark AI Slides

Thread ID — Ключ к контексту

Конфигурация вызова

```
config = {
  "configurable": {
    "thread_id": "ticket-123"
  }
}
```

Уникальный идентификатор сессии передается в каждом запросе

Принципы работы

Изоляция сессий (Session Isolation)

`thread_id` работает как первичный ключ (Primary Key) в базе данных чекпоинтов. Он строго разделяет диалоги разных пользователей, предотвращая утечку контекста.

Enterprise идентификаторы

В реальных системах вместо случайных UUID используйте бизнес-сущности: `ticket_id`, `user_session_id` или `process_id`. Это критически важно для логирования и аудита.

Механизм восстановления (Hydration)

Если вызвать **`app.invoke(...)`** с существующим ID, LangGraph автоматически: 1) Загрузит последний snapshot из Postgres. 2) Восстановит переменные стейта. 3) Продолжит выполнение с прерванного места.

© 2026 GenSpark AI Slides

HITL: Паттерн утверждения человеком

Блок 2: Human-in-the-Loop

⚠ Проблема доверия (Trust)

Агенты могут "галлюцинировать" или принимать неверные решения. Критические действия требуют премодерации.

Критические сценарии

Внешние коммуникации
Отправка Email клиентам, публикация постов, ответы в чате поддержки.

Изменение данных (Write Ops)
Выполнение SQL (INSERT/DELETE), транзакции API, удаление файлов.

МЕХАНИЗМ РЕШЕНИЯ

Использование `interrupt_before` или `interrupt_after` позволяет "заморозить" граф перед опасным шагом.

graph_builder.py

```python
# Определяем узлы графа
builder.add_node("planner", planner_node)
builder.add_node("action_node", execute_sql)

# Строим ребра
builder.add_edge("planner", "action_node")

# ⚡ КЛЮЧЕВОЙ МОМЕНТ: Прерывание
# Остановиться ПЕРЕД выполнением узла 'action_node'
app = builder.compile(
    checkpointer=checkpointer,
    interrupt_before=["action_node"]
)

# При запуске:
# 1. Выполнится 'planner'
# 2. Граф увидит, что следующий шаг 'action_node'
# 3. Сохранит checkpoint и ОСТАНОВИТСЯ
# 4. Вернет управление (как будто закончился)
```

LangGraph 1.0.7 API `graph.compile(...)`

Жизненный цикл HITL

Блок 2: Human-in-the-Loop

1

Graph Execution

Reach Node

Граф выполняет шаги и доходит до узла `action_node`, помеченного как прерываемый.

2

Detection

Interrupt!

Checkpointer обнаруживает настройку `interrupt_before`.

3

Persistence

Save State

Полный снимок состояния (Snapshot) сохраняется в Postgres с текущим `thread_id`.

4

Suspension

Stop & Wait

Выбрасывается исключение `GraphInterrupt`. Процесс останавливается и освобождает ресурсы.

5

Human Action

Resume Command

Ожидание внешнего сигнала (API call) для продолжения: Approve, Reject или Edit.

Важно для архитектора

В состоянии "Stop & Wait" агент не потребляет CPU/GPU ресурсы. Состояние хранится в БД, а не в памяти процесса. Это позволяет держать миллионы "ждущих" агентов.

© 2026 GenSpark AI Slides

Работа оператора (Resume)

Блок 2: Human-in-the-Loop

ИНТЕРФЕЙС ОПЕРАТОРА

Взгляд Человека

Оператор через UI видит, что агент встал на паузу и запрашивает разрешение на критическое действие.

```
pending action

Action: Execute SQL
"DROP TABLE users"
⚠ Waiting for approval...
```

Варианты реакции

Вариант A: Reject — Отмена

Оператор полностью отменяет действие. Агент получает сигнал об ошибке или отказе и может попробовать другой путь (или завершить работу).

Вариант Б: Approve — Resume

Оператор разрешает выполнение. Вызывается `app.invoke(None, config)`. Граф продолжает выполнение с того места, где остановился.

Вариант B: Edit — Patching

Оператор видит ошибку (DROP TABLE) и исправляет её на безопасный запрос вручную, обновляя стейт перед возобновлением.

Редактирование Состояния (State Patching)

Блок 2: Human-in-the-Loop

patch_state.py — Python 3.12

```python
# 1. Получаем текущий стейт для анализа (во время Interrupt)
state = await app.aget_state(config)

# 2. Меняем "плохой" SQL запрос на корректный (исправляем галлюцинацию)
await app.update_state(
    config,
    {"sql_query": "SELECT * FROM users LIMIT 10"}, # Патч (значение)
    as_node="generator_node" # Якобы это сгенерировал генератор
)

 # 3. Возобновляем выполнение уже с исправленными данными
 await app.ainvoke(None, config=config)
```

Inspection (aget_state)

Позволяет оператору или системе увидеть текущий снимок памяти (Snapshot) перед вмешательством.

State Mutation

update_state изменяет значения переменных в графе "на лету", исправляя ошибки без перезапуска.

Node Impersonation

as_node указывает графу, что изменение пришло от конкретного узла, сохраняя логику переходов (Edges).

© 2026 GenSpark AI Slides

Паттерн Time Travel (Replay)

Блок 2: Human-in-the-Loop

Зачем нужен "Time Travel"?

Агенты — сложные системы. Когда что-то идет не так (например, агент нагрубил клиенту), нам нужно не просто видеть результат, а "проиграть" ход мыслей модели шаг за шагом.

Сценарии использования

Post-Mortem Debugging
Анализ инцидентов. Почему Router выбрал этот путь? Что именно сгенерировал Qwen3?

Audit & Compliance
Юридическое доказательство того, какие данные были у агента в момент принятия решения.

КАК ЭТО РАБОТАЕТ

LangGraph хранит иммутабельную историю версий состояния. Вы можете запросить `aget_state_history` и получить итератор по всем шагам выполнения для конкретного `thread_id`.

debug_history.py

```python
# Конфигурация проблемного треда
config = {"configurable": {"thread_id": "incident-2026-01-26"}}

# Получаем ВСЮ историю изменений State
history = []
async for state in app.aget_state_history(config):
    history.append(state)

# Анализируем шаги (от последнего к первому)
for i, step in enumerate(history):
    print(f"Step {i}: Node {step.metadata.get('source')}")
    print(f"Memory: {step.values.get('messages')[-1].content}")
    print("---")

# Вывод:
# Step 0: Node 'generator' → "Ты тупой бот!" (Error)
# Step 1: Node 'router' → Route to 'chitchat'
# Step 2: Node 'input' → User: "Привет"
```

LangGraph 1.0.7 API `app.aget_state_history(...)`

Forking (Ветвление истории)

МЕХАНИЗМ FORKING

Параллельные реальности

Мы можем взять любой `Checkpoint` из истории диалога и запустить от него новую ветку выполнения.

Оригинальное состояние остается неизменным, создается изолированная копия контекста для экспериментов.

Сценарии применения

1. Сценарии «Что если» (What If Analysis)

Анализ альтернативных путей в графе. "А что, если бы Router отправил запрос не в TechSupport, а в Sales?". Можно принудительно изменить решение узла и посмотреть, как поведет себя агент дальше.

2. A/B Тестирование Промптов

Тестирование изменений на **реальных исторических данных**. Берем стейт сложного кейса пользователя и запускаем его через новую версию промпта (v2). Сравниваем результат v1 и v2 без необходимости симулировать диалог с нуля.

3. Отладка и Regression Testing

Если агент совершил ошибку на 5-м шаге, разработчик может откатиться на 4-й шаг, исправить код ноды и перезапустить выполнение именно с этого момента, сохраняя весь контекст предыдущих шагов.

Демонстрация в Langfuse

trace_id: tr_8f92a1 Live

Start: Agent SUCCESS

Node: Search SUCCESS

Node: Approval INTERRUPTED

2 hours later...

Span: Human Review APPROVED

Node: Generate RUNNING

Этапы выполнения в трейсе

1. Прерывание (Interrupted) Status

Граф доходит до узла с `interrupt_before`. LangGraph сохраняет контрольную точку (checkpoint) в Postgres и останавливает выполнение процесса. Статус трейса меняется на "Interrupted".

2. Проверка (Human Review) Action

Оператор видит запрос в интерфейсе. В трейсе может появиться отдельный спан, фиксирующий время ожидания и решение человека (Approved/Rejected) вместе с метаданными.

3. Возобновление (Resumed) Event

После вызова `app.invoke(Command(resume=...))` граф загружает состояние и продолжает выполнение со следующего узла (Generate).

Зачем нам Async?

ПРОБЛЕМА I/O BOUND

LLM — это медленные операции ввода-вывода. Основное время тратится не на CPU вашего сервера, а на ожидание ответа от сетевого API или GPU-кластера.

GIGACHAT / GPT-4

2-5 sec

Средняя задержка на генерацию среднего ответа.

QWEN3-80B (LOCAL)

1-3 sec

Inference time на A100/H100 при нагрузке.

TOTAL SPEEDUP

5x faster

При параллельном запуске 5 агентов одновременно.

EXECUTION TIMELINE COMPARISON (5 TASKS)

⚠ `Blocking calls (time.sleep) kill the Event Loop. Always use await.`

Async в LangGraph: принципы

SYNC BLOCKING

Sequential

Синхронный код блокирует поток выполнения. Пока модель отвечает (3 сек), сервер "зависает" и не может принять новые запросы.

ASYNC NON-BLOCKING

Event Loop

LangGraph использует `asyncio`. Один поток обрабатывает тысячи агентов, переключаясь между ними во время ожидания I/O (сети).

```
async def node(state):
# Non-blocking call
  res = await llm.ainvoke(...)
  return res
```

Правила разработки (Best Practices)

1. Определяйте узлы как async def

Все функции узлов графа (nodes) должны быть асинхронными. LangGraph может запускать и синхронные функции, но они рискуют заблокировать поток при высокой нагрузке.

2. Используйте Ainvoke

Забудьте про `llm.invoke()` в продакшене. Только `await llm.ainvoke(...)`. Это позволяет Event Loop переключиться на другую задачу, пока LLM генерирует токены.

3. Избегайте блокировок (Blocking Calls)

`time.sleep(5)` "убьет" производительность всего сервера. Используйте `await asyncio.sleep(5)`. Тяжелые вычисления CPU выносите в отдельные потоки или процессы.

© 2026 GenSpark AI Slides

Паттерн Fan-Out (Map)

Параллельное распределение

Паттерн Map (или Fan-Out) позволяет динамически создавать множество параллельных задач на основе данных, полученных в ходе выполнения графа.

Ключевой сценарий: "Brainstorming"

Задача
"Сгенерируй 3 варианта заголовка для статьи".

Реализация
Узел `brainstorm` возвращает список команд `Send`. LangGraph запускает целевой узел (writer) для каждого элемента списка **одновременно**.

МЕХАНИКА MAP-REDUCE

Это первая часть паттерна Map-Reduce. Мы "разбрасываем" (Map) задачи, чтобы потом собрать их результаты (Reduce) в следующем узле.

nodes/brainstorm.py

```python
from langgraph.types import Send
from typing import Literal

async def brainstorm_node(state: State): # 1. Получаем тему из стейта     topic = state["topic"] # 2. Определяем, сколько вариантов нужно     count = 3

    # ⚡ FAN-OUT: Динамическая генерация
    # Вместо одного перехода, возвращаем список Send
    return [
            Send(
                "writer_node", # Целевой узел
                {"topic": topic, "idx": i} # Payload
            )
            for i in range(count)
        ]

# Результат: 3 параллельных вызова writer_node
# writer_node(topic="...", idx=0)
# writer_node(topic="...", idx=1)
# writer_node(topic="...", idx=2)
```

LangGraph 1.0.7 / Send API

`List[Send]`

Parallel Node Execution

МЕХАНИЗМ

LangGraph автоматически запускает узлы параллельно, если между ними нет прямых зависимостей данных.

> Пример: Генерация 3 вариантов текста разными промптами может идти одновременно.

ГЛАВНАЯ ПРОБЛЕМА

Тяжелые модели (80B+) быстро забивают GPU VRAM при батчинге, вызывая OOM (Out of Memory) или замедление из-за свопинга.

Выбор модели для параллелизма

Qwen3-80B

Main

VRAM USAGE
Очень высокое

LATENCY
1.5 - 3.0 сек

Риск переполнения памяти при 3+ потоках.

РЕКОМЕНДОВАНО

Qwen3-30B-A3B

Worker

THROUGHPUT
Максимальный

LATENCY
~0.4 - 0.8 сек

CONCURRENCY
5-10 инстансов

Идеально для Fan-Out операций: быстро запускаем 5 агентов, получаем результат мгновенно.

Паттерн Fan-In (Reduce)

Блок 3: Асинхронность

Агрегация результатов

Паттерн Fan-In (или Reduce) используется для сбора результатов от множества параллельно выполняемых задач. Aggregator Node ожидает завершения всех входящих веток перед запуском.

МЕХАНИКА РАБОТЫ

Wait for All

Aggregator запускается только когда все Writer закончили

State Collection

Используем списки: `drafts: Annotated[list, add_messages]`

PARALLEL EXECUTION FLOW

Async Writer #1 — Draft A — Done (1.2s)

Async Writer #2 — Draft B — Done (2.5s)

Async Writer #3 — Draft C — Done (0.8s)

AGGREGATOR NODE

Reduce / Merge

```
state.drafts = [
  "Draft A
  content...",
  "Draft B
  content...",
  "Draft C
  content..."
]
```

Условная асинхронность (Router)

Блок 3: Асинхронность

Conditional Edges

В LangGraph маршрутизация (Routing) — это не просто `if/else`. Это логический узел, который анализирует состояние и динамически определяет следующий шаг (или шаги).

```
// Router Function Signature
async def router(state) → list[str]:
```

Механизмы принятия решений

Асинхронные вычисления сигналов

Роутер не блокирует поток. Мы запускаем эмбеддер (например, **Deepvk-bge-m3**) или классификатор интентов асинхронно через `await`, чтобы получить вектор запроса и решить, куда идти дальше.

Множественная активация путей

Условное ребро может вернуть не одну строку, а список узлов: `["audit_log", "answer_user"]`. Это инициирует **параллельное** выполнение выбранных веток графа.

Фильтрация контекста

Роутер также может решать, какую часть State передать дальше. Например, ветка "Legal Check" получит только последние сообщения, чтобы сэкономить токены и ускорить обработку.

Map-Reduce через Send API

node_map.py — Python 3.12

```python
from langgraph.types import Send

# Узел маршрутизатора (Map Phase)
def map_node(state: State):
    urls = state["urls"] # Динамический список задач (неизвестной длины)

    # Fan-Out: Создаем параллельные задачи через Send
return [
        Send("scrape_node", {"url": u, "attempt": 1})
         for u in urls
     ]
 # Узел воркера (Execute Phase)
 async def scrape_node(state: ScrapeState):
     # Запустится N раз параллельно
 content = await tavily.asearch(state["url"])
```

Dynamic Fan-Out

Send API позволяет ветвить процесс выполнения динамически, создавая столько задач, сколько элементов в списке, без жесткого кодирования ребер.

Parallel Workers

LangGraph автоматически запускает воркеры параллельно (через asyncio.gather под капотом).

Implicit Reduce

Сборка результатов (Reduce) происходит автоматически через механизм Annotated[list, add_messages] в общем стейте.

Batch Processing

SEQUENTIAL LOOP

Slow

Обычный цикл `for` вызывает `invoke` последовательно. Общее время = сумма времени всех запросов (N × T).

BATCH API

Fast

Метод `.batch()` запускает обработку списка входных данных параллельно. Общее время ≈ время самого медленного запроса.

Ключевые аспекты

Sync vs Async API

.batch(): использует ThreadPoolExecutor. Подходит для синхронного кода.
.abatch(): использует `asyncio.gather`. Идеален для FastAPI/Asyncio приложений. Не блокирует Event Loop.

High Throughput

Позволяет обработать 100+ запросов пользователей одновременно одной строкой кода. Применяется для массовой классификации тикетов или оценки датасетов.

Важно: Rate Limits

Убедитесь, что ваш vLLM сервер (память GPU) или лимиты GigaChat (RPM/TPM) выдержат нагрузку. Используйте параметр `config={"max_concurrency": 5}` для защиты.

© 2026 GenSpark AI Slides

Race Conditions (Гонка данных)

Проблема перезаписи

Если 3 параллельные ноды пытаются обновить один и тот же ключ в State (например, `summary`), победит последний записавший. Данные других нод будут потеряны.

Стратегии решения

Иммутабельное добавление
Вместо перезаписи строки `str` используйте список `list` и операцию добавления.

Custom Reducers
Определите функцию-редьюсер, которая знает, как объединять конфликтующие данные.

СОВЕТ АРХИТЕКТОРА

"Никогда не используйте простые типы (str, int) для ключей, которые обновляются параллельно. Всегда используйте коллекции."

state_definition.py

```python
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages
import operator

# ❌ ПЛОХО: Race Condition гарантирован
class BadState(TypedDict):
    summary: str # Перезапишется!

# ✅ ХОРОШО: Безопасно для параллелизма
class GoodState(TypedDict):
    # LangGraph автоматически объединит списки сообщений
    messages: Annotated[list, add_messages]

    # Или для произвольных данных:
    drafts: Annotated[list[str], operator.add]

# При параллельном возврате {"drafts": ["A"]} и {"drafts": ["B"]}
# Результат будет: drafts: ["A", "B"]
```

Python Type Hints `Annotated[Type, reducer]`

Ограничение параллелизма

Блок 3: Асинхронность

Риск перегрузки (Overload)

Неконтролируемый запуск тысяч параллельных задач (unbounded fan-out) может исчерпать память, файловые дескрипторы или привести к бану по API Rate Limits.

Механизмы контроля

Семафоры (Semaphores)
Ограничивают количество одновременно выполняемых корутин. Идеально для внешних API.

Воркеры (Worker Pools)
LangGraph позволяет настроить максимальное число потоков выполнения через конфигурацию `max_concurrency`.

BEST PRACTICE

Всегда оборачивайте вызовы LLM в семафор. Например, для GigaChat лимит может быть 10 RPS, а для локальной Qwen3 — объем VRAM.

async_utils.py

```python
import asyncio
from langchain_core.runnables import RunnableConfig

# Глобальный ограничитель: макс 5 одновременных запросов
llm_semaphore = asyncio.Semaphore(5)

async def safe_llm_call(prompt: str):
    # Ждем разрешения войти в критическую секцию
    async with llm_semaphore:
        # В этот блок войдут только 5 тасок одновременно
        return await llm.ainvoke(prompt)

# Внутри узла графа
async def process_batch(state):
    tasks = [safe_llm_call(item) for item in
state["items"]]
    # Запустим хоть 100 задач, но выполняться будут по 5
    results = await asyncio.gather(*tasks)
    return {"results": results}
```

Python asyncio Standard Library

`asyncio.Semaphore`

Ошибки в асинхронном мире

ПРОБЛЕМА

Частичный сбой

Представьте, что запущено 5 параллельных веток. 4 отработали успешно, а 1 упала с `Exception`.

"Что делать с общим состоянием графа? Прерывать всё или пытаться спасти результат?"

3 Стратегии обработки

1. Fail Fast (Уронить граф)

Бросаем исключение наверх. Весь процесс останавливается. Подходит для критических данных, где частичный результат неприемлем (например, финансовая транзакция).

2. Best Effort (Игнорировать)

Логируем ошибку, но граф продолжает выполнение с результатами успешных веток. Подходит для задач типа "поиск новостей" — если один источник отвалился, не страшно.

3. Fallback Value (Дефолт)

Возвращаем безопасное значение-заглушку вместо ошибки (например, пустой список или "N/A"). Граф получает полные данные, но часть из них — фиктивная.

Retry Policies

Блок 4: Resilience

graph_builder.py — Python 3.12

```python
from langgraph.prebuilt import RetryPolicy

builder.add_node(
    "api_call",
    node_func,
    # Конфигурация политики повторов (работает и внутри async)
    retry=RetryPolicy(
        max_attempts=5,
        backoff_factor=2, # Экспоненциальный рост: 1s, 2s, 4s...
         retry_on=ConnectionError # Игнорировать ValueError и др.
     )
 )
```

Exponential Backoff

Увеличение интервала между попытками снижает нагрузку на упавший сервис ("thundering herd" problem).

Selective Retry

Повторяем только transient errors (сеть, 503). Логические ошибки (400, ValueError) повторять бессмысленно.

Built-in Stability

LangGraph обрабатывает ретраи на уровне оркестратора, освобождая код узла от try-except блоков.

© 2026 GenSpark AI Slides

Circuit Breaker (Предохранитель)

Блок 4: Resilience

Проблема каскадных сбоев

Если внешний API (GigaChat, OpenAI) лежит и возвращает 500 Error, постоянные Retry только усугубляют нагрузку и замедляют систему.

Логика работы

Состояние Closed (Норма)
Все запросы проходят. Считаем ошибки. Если ошибок > порога (напр. 5), переходим в Open.

Состояние Open (Сбой)
Запросы мгновенно отклоняются или перенаправляются на Fallback (Qwen3-30B) без попытки вызова API.

РЕАЛИЗАЦИЯ

Храним статус здоровья сервиса в **Redis** с TTL (например, 60 секунд). Это делает "предохранитель" распределенным между всеми воркерами.

nodes/safe_generation.py

```
async def safe_llm_call(state): # 1. Проверяем, открыт ли
прерыватель is_open = await
redis.get("circuit:gigachat:open") if is_open: # Fast
Fallback: сразу идем к локальной модели print("⚠️ Circuit
Open: Switching to local Qwen") return await
local_llm.ainvoke(state["messages"])

try:
# 2. Пытаемся вызвать основной сервис
response = await gigachat.ainvoke(state["messages"])
await redis.delete("circuit:gigachat:errors") # Reset
return response

except Exception as e:
# 3. При ошибке увеличиваем счетчик
errors = await redis.incr("circuit:gigachat:errors")

if errors ⩾ 5:
# 4. Открываем прерыватель на 60 сек
await redis.setex("circuit:gigachat:open", 60, "1")

return Command(goto="fallback_node")
```

Redis State

`redis.setex(key, ttl, value)`

Тайм-ауты (Timeouts)

Блок 4: Resilience

nodes.py — Python 3.12

```python
import asyncio
from langgraph.types import Command

async def generate_node(state):
    try:
        # Жёсткий лимит 30 секунд на ответ модели
        response = await asyncio.wait_for(
            llm.ainvoke(state["messages"]), timeout=30.0
        )
        return {"messages": [response]}

    except asyncio.TimeoutError:
        # На тайм-аут — переход в fallback, минуя обычный маршрут
        return Command(goto="fallback_node", update={"error": "Timeout"})
```

Hard Limits

Асинхронные задачи могут "висеть" вечно. `wait_for` гарантирует освобождение ресурсов event loop'а.

Dynamic Routing

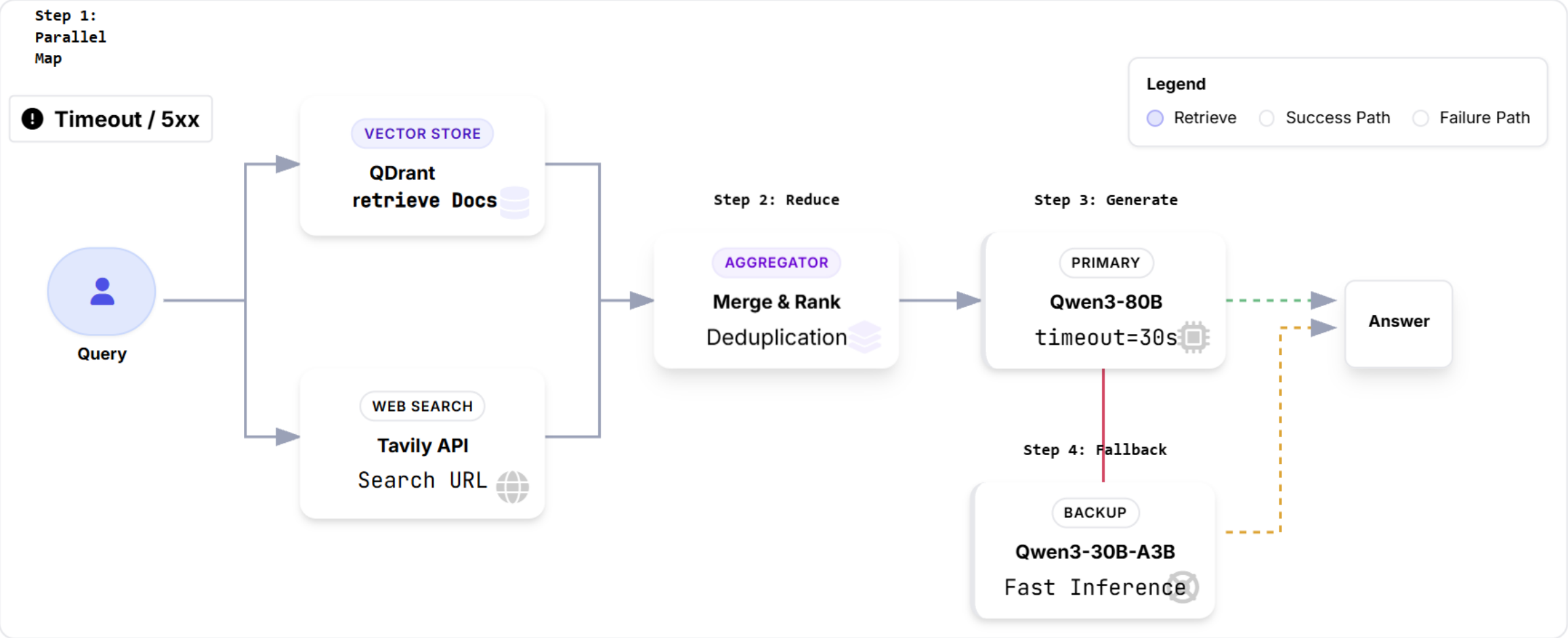
Использование `Command(goto=...)` позволяет мгновенно изменить поток выполнения при ошибке.

UX Protection

Пользователь получает быстрый ответ от резервной модели (Fallback) вместо бесконечного ожидания ответа.

Архитектура Robust RAG

Пример реализации устойчивого пайплайна (Resilient Pipeline). Использует параллельный поиск для скорости, объединение результатов для полноты и механизм Fallback для обработки сбоев основной модели.



Итоговый стек Resilience

Рекомендованная архитектура для создания отказоустойчивых Enterprise-агентов в 2026 году.

Storage Layer

`AsyncPostgresSaver`

Использование PostgreSQL с JSONB для надежного хранения чекпоинтов. Обеспечивает транзакционную целостность и возможность восстановления после сбоев.

Model Strategy

`Qwen3-80B (Main)` `Qwen3-30B-A3B (Fallback)`

Двухуровневая стратегия: мощная модель для сложных задач и быстрая/дешевая модель для простых задач, параллелизма и резервного переключения.

Async Runtime

`asyncio` `Semaphores` `Send API`

Полностью асинхронное ядро для I/O-bound операций. Управление конкурентностью через семафоры и динамический маппинг задач через Send API.

Observability

`Langfuse Tracing` `Alerts`

Полная прозрачность выполнения графа. Отслеживание трейсов, статусов прерывания (HITL) и алертинг при срабатывании Circuit Breaker.

Задание: News Digest Agent

Практическая работа

ЦЕЛЬ ЗАДАНИЯ

Реализовать агента новостного дайджеста, используя паттерны **Map-Reduce** для параллельного поиска и **Human-in-the-Loop** для премодерации контента. Стек: Qwen3-30B-A3B + Tavily.

01 Input / Ingestion — `State`

Принимает тему от пользователя (например, "AI Trends 2026"). Инициализирует `State`.

02 Map: Query Generation — `Qwen3-30B`

Генерирует 3 различных поисковых запроса. Возвращает список `Send("search", ...)`.

03 Parallel Execution — `Async/Tavily`

Выполняет поиск параллельно. Использует `await tavily.search()`.

04 Reduce / Aggregate — `Reducer`

Собирает результаты поиска в единый контекст. Удаляет дубликаты.

05 Human Review (HITL) — `Interrupt`

Показывает черновик. Ждет команды `Command(resume="Approve")`.

06 Generate Post — `Final Output`

Формирует финальный пост в Markdown. Сохраняет в БД.

Ожидаемый результат: Langfuse Trace с параллельным выполнением и желтым статусом "Interrupted".